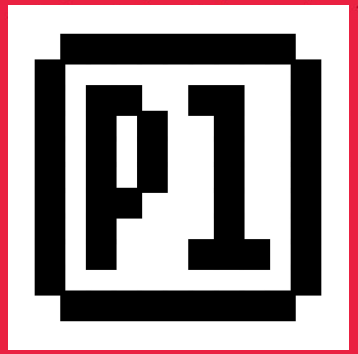
Estructuras de control

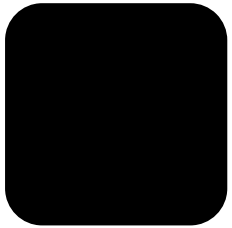


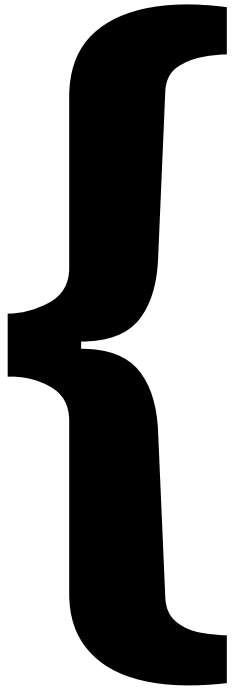
Martín René Vilugrón
mrvilugron@unrn.edu.ar

UNRN
Universidad Nacional
de Río Negro

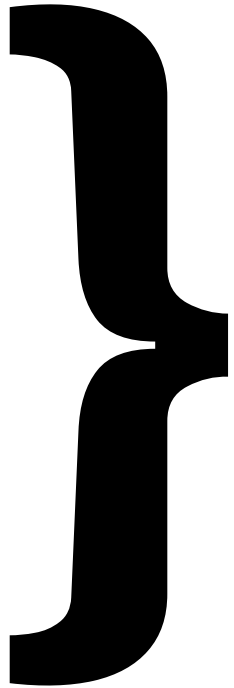
r33

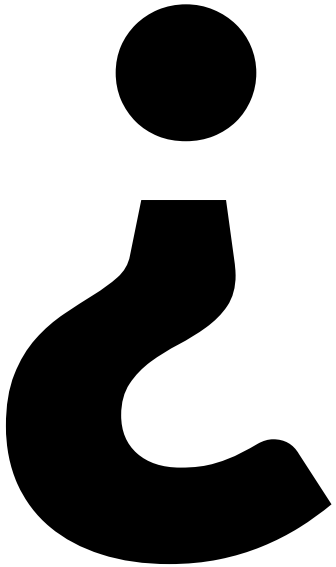
03 / 2026





Repasando





Cual es la estructura mínima de un programa en C



El “Hola Mundo C.”

```c
#include <stdio.h>

int main(void)
{
    printf("Hola mundo C. \n");
    return 0;
}
```

Operadores relacionales

UNRN Universidad Nacional de Río Negro

```
n > m
n < m
n >= m
n <= m
n == m
n != m
```

} int





Dan como resultado un número entero

`int`

0 es falso
1 es verdadero*

*Cualquier otra cosa que no sea cero es verdadero

Operadores lógicos

para condiciones más complejas

UNRN Universidad Nacional de Río Negro



negación
y lógico
o lógico

Un ejemplo rápido

al que volveremos en instantes nada más

```
(nota >= 4) && (nota<7)
```



Al usar condiciones más complejas

```
bool esta_aprobado = false;
…
esta_aprobado = (nota >= 4) && (nota < 7);
```

Podemos tener *una mejor idea* de cual es el propósito de la condición

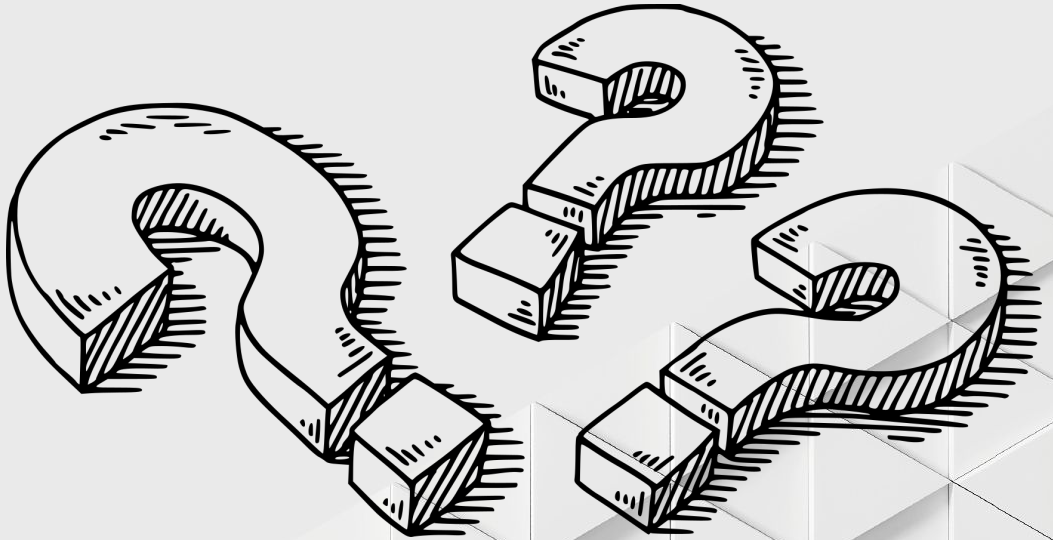

¿Preguntas?

Y esto...

¿Para qué?

Tomar decisiones

(condicionales)



Estructuras de control

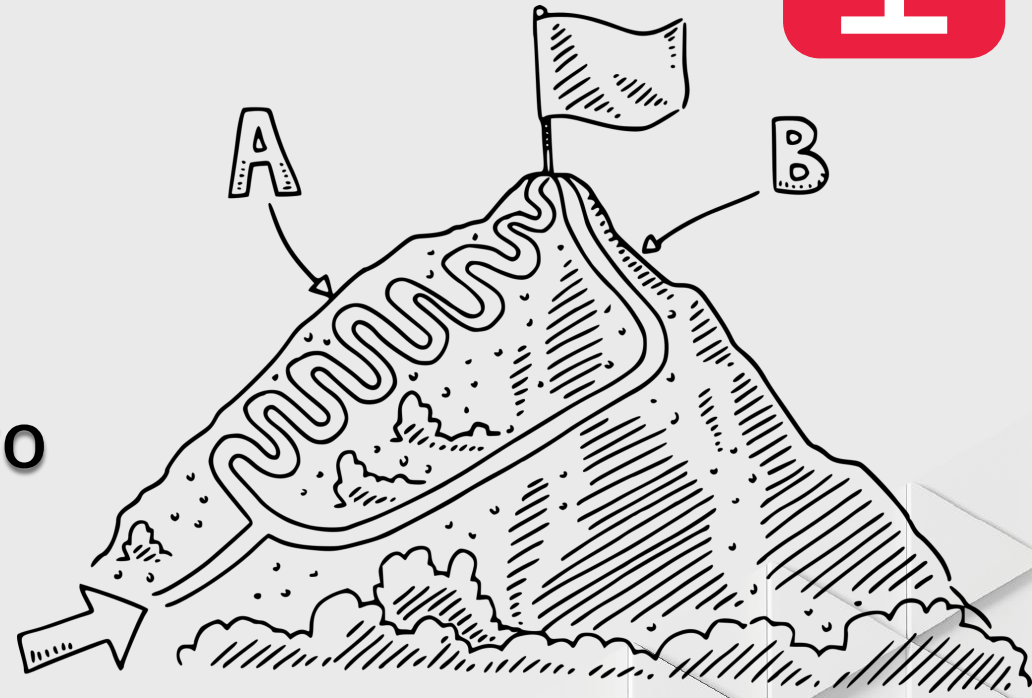
y el estilo a usar

UNRN Universidad Nacional de Río Negro

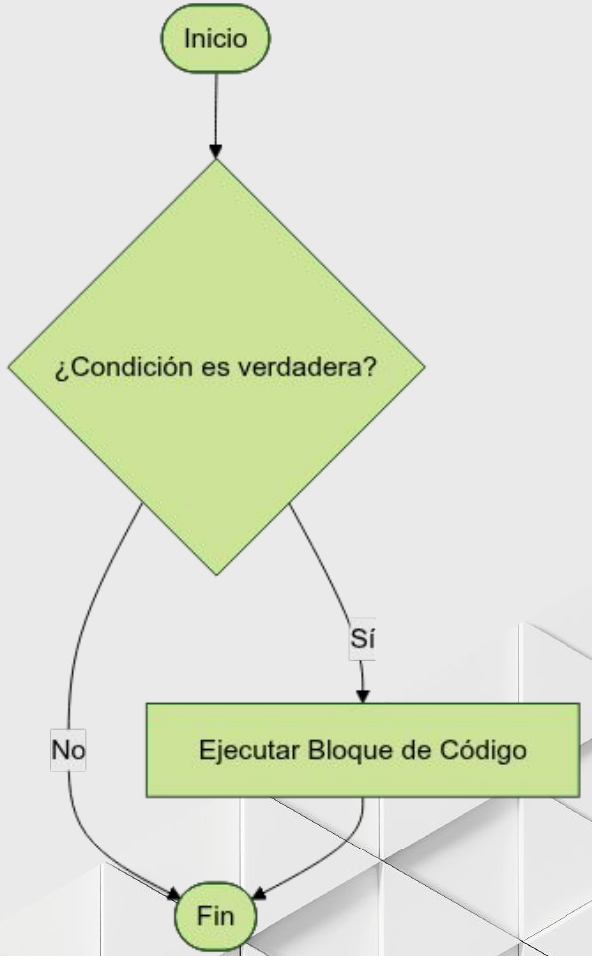
1

if simple

un camino alternativo

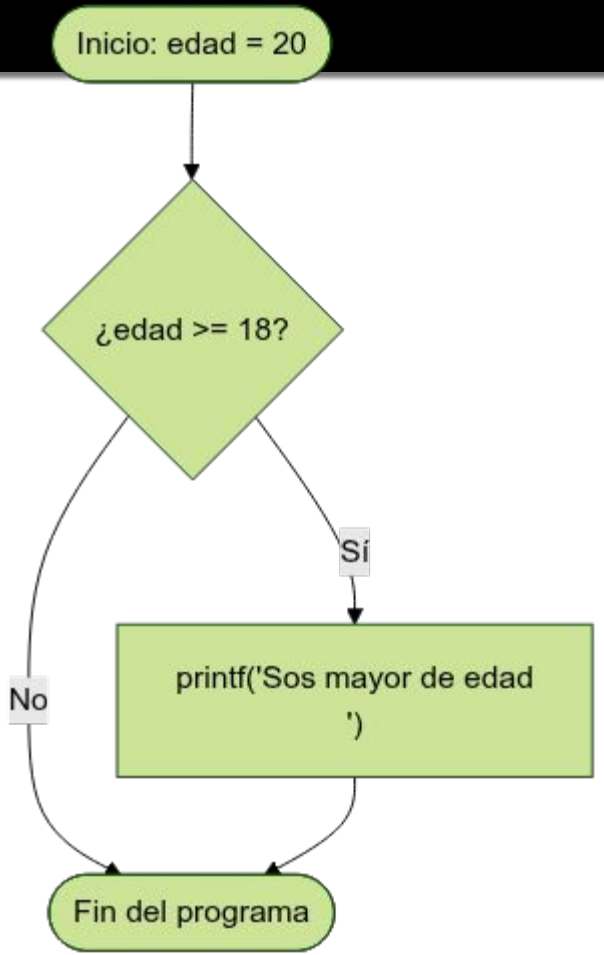


```
if (condicion) {
    bloque verdadero;
}
```



Obtenemos una salida

```
if (edad >= 18) {
    printf(“Mayor de\n”);
}
```



La condición puede ser una variable

```
int esta_aprobado = (nota >= 4) && (nota < 7);

if (esta_aprobado)
{
    printf("Aprobado\n");
}
```

Vamos a volver a esto más adelante

Pero mejor si es booleana

```
#include <stdbool.h>

bool esta_aprobado = (nota >= 4) && (nota < 7);

if (esta_aprobado)
{
    printf(“Aprobado\n”);
}
```

¡A la terminal!



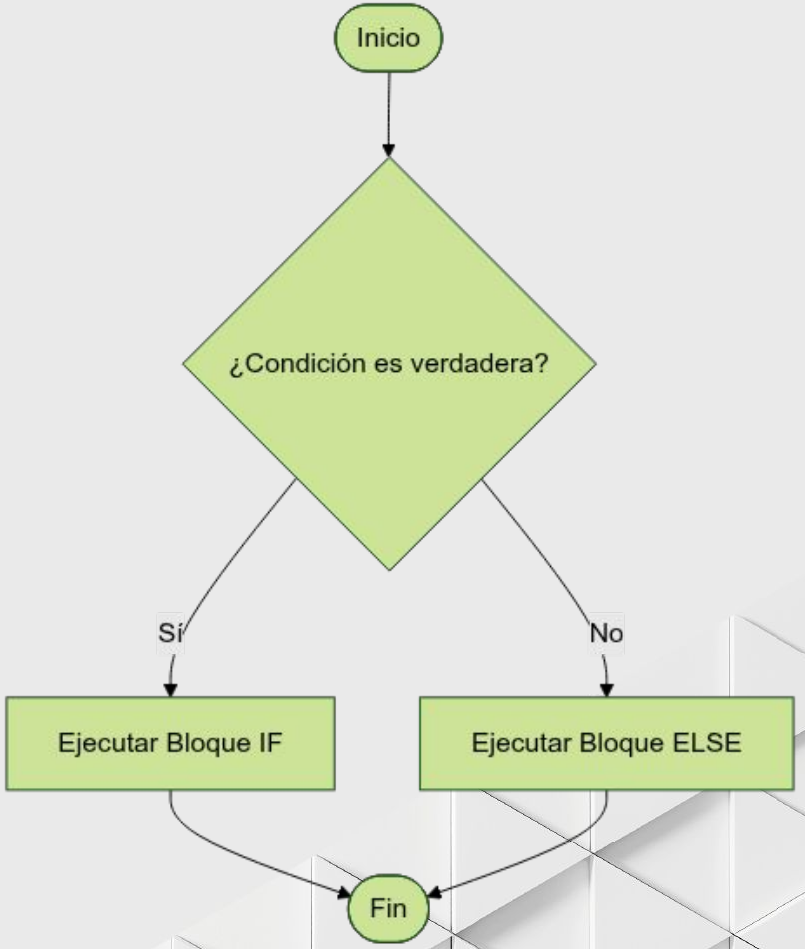
2

if/else

dos caminos excluyentes

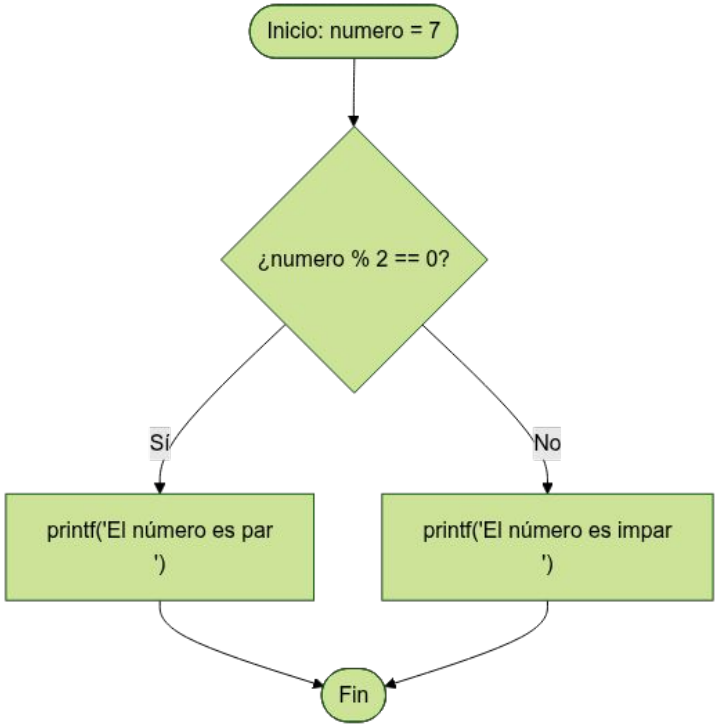


```
if (condición){
    bloque verdadero;
} else {
    bloque falso;
}
```



Obtenemos dos salidas diferentes

```
int numero = 7;
if (numero % 2 == 0) {
    printf("Es par\n");
} else {
    printf("Es impar\n");
}
```



¡A la terminal!



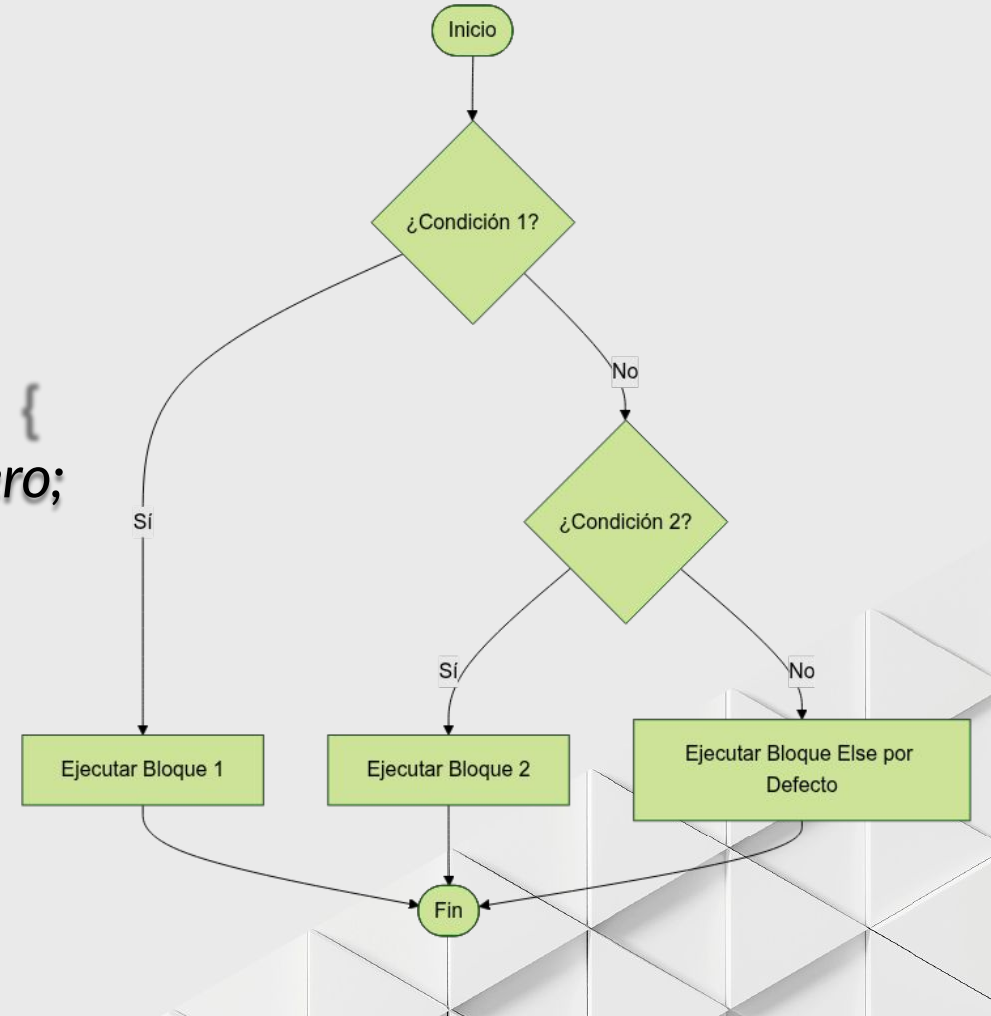
3

if/else if/else

varios caminos

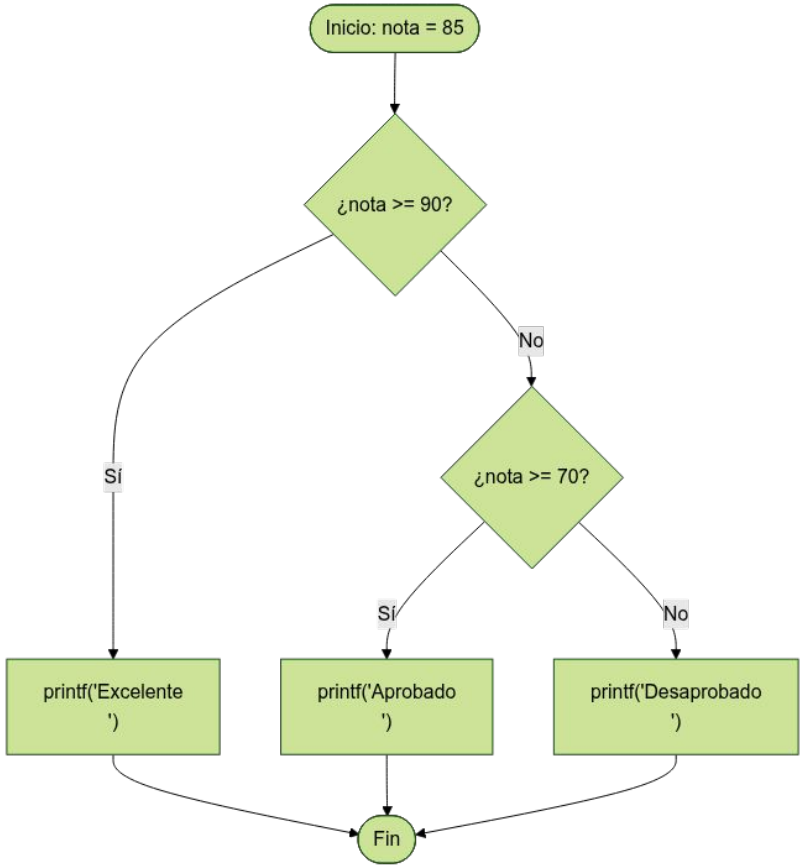


```
if (condición) {
    bloque condición verdadero;
} else if(otra_condición) {
    bloque otra_condición verdadero;
} else {
    bloque falso de los otros
}
```



Obtenemos tres salidas diferentes

```
int nota = 85;
if (nota >= 90) {
    printf(“Excelente”);
} else if (nota >= 70) {
    printf(“Aprobado”);
} else {
    printf(“desaprobado”);
}
```



¡A la terminal!




¿Preguntas?

Cuestiones a tener en cuenta:

UNRN Universidad Nacional de Río Negro

Todos los bloques van con llaves

y *si es posible* en línea propia

¡A la terminal!



Error fino gramatical



```
int x = 5;
if (x = 0) {
    printf("cero\n");
} else {
    printf("no cero\n");
}
```

Asignación en lugar de comparación

¡A la terminal!



{ **Se ejecuta solo un camino** }

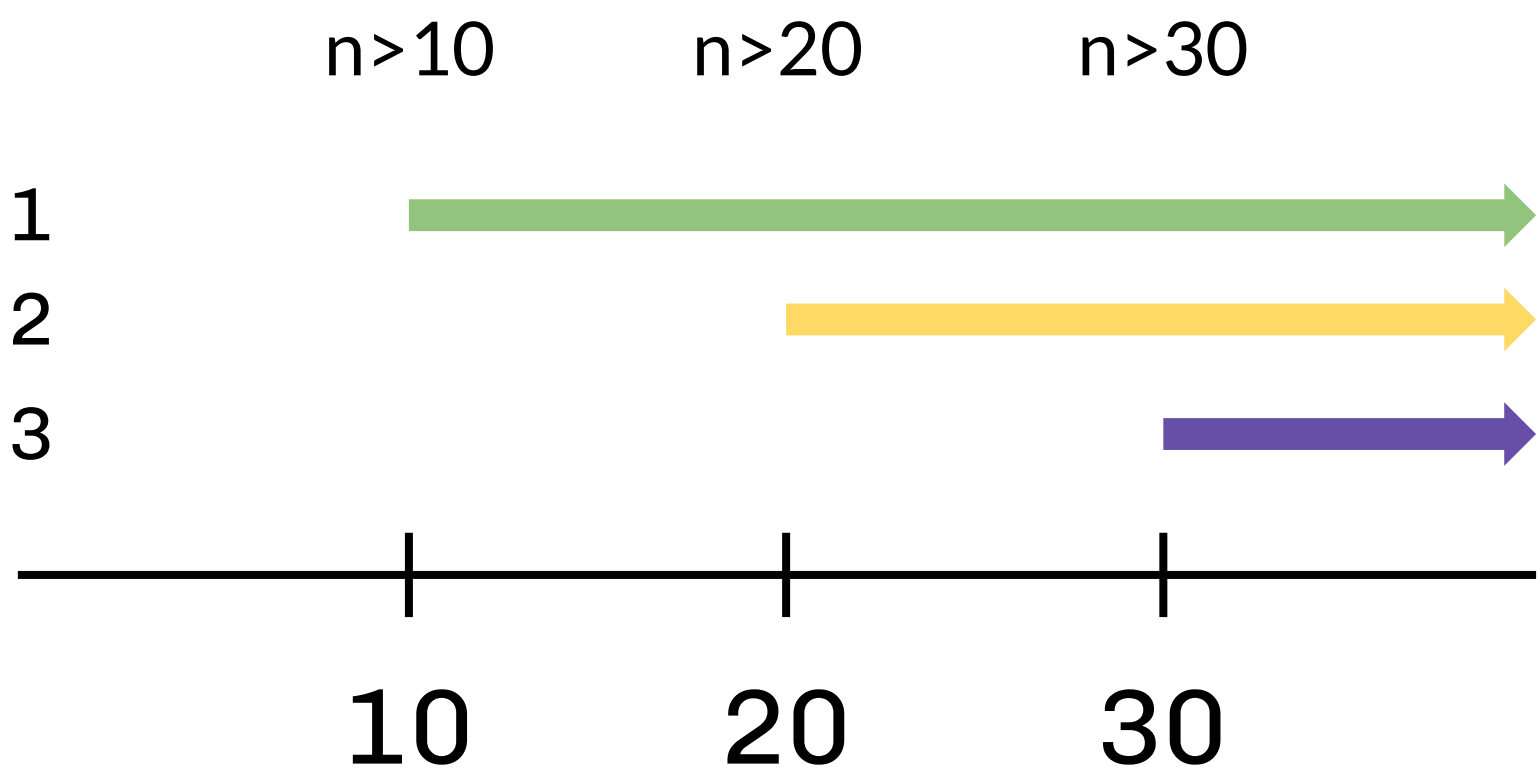
{ Se ejecuta solo el primero que coincida }

¡Cuidado con las superposiciones!

```
int numero = scanf("%d");

if (numero >= 0){
    printf("Es cero");
} else if (numero < 4) {
    printf("menor a 4");
} else {
   printf("que va aca?");
}
```



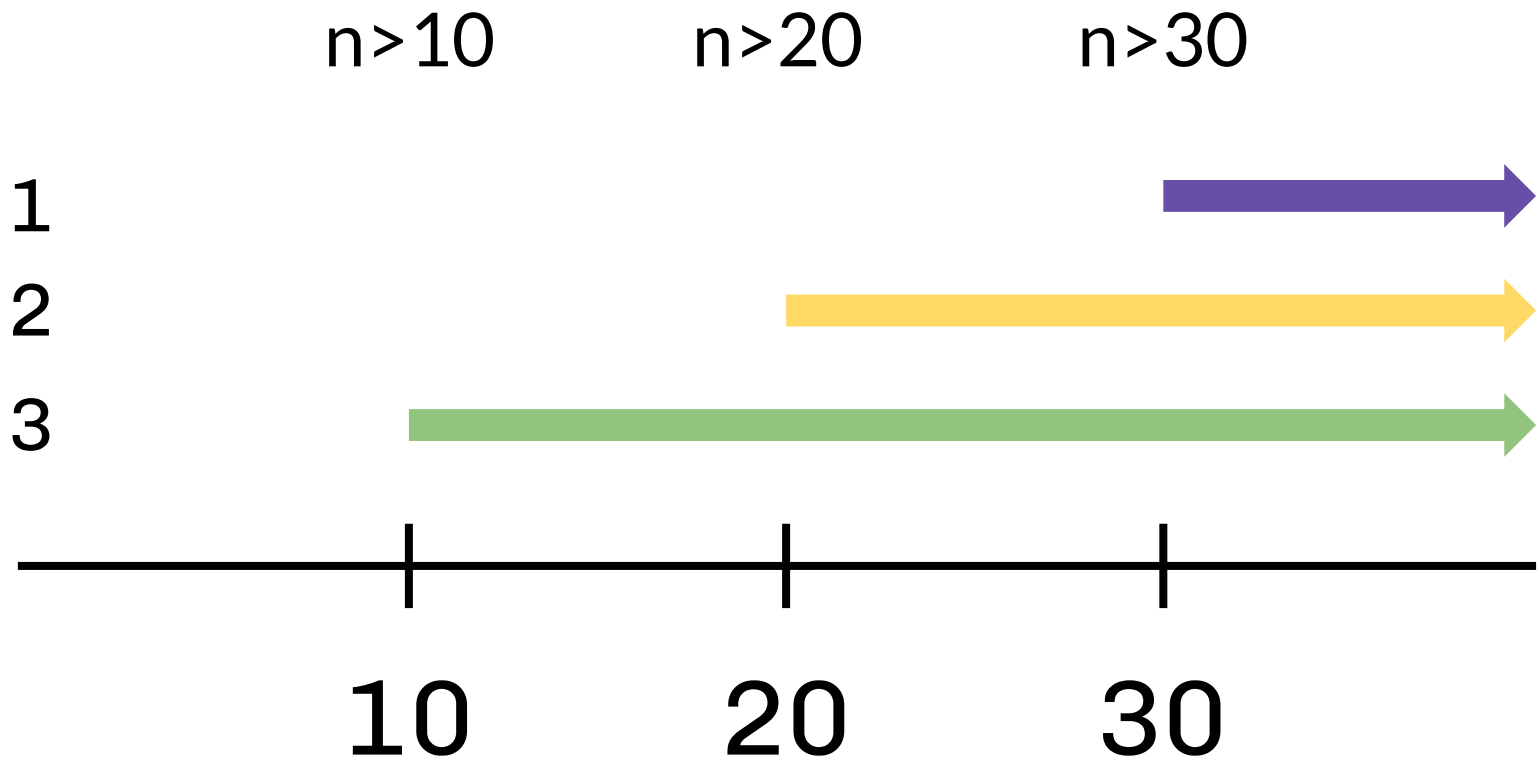

n>10
n>20
n>30
1
2
3
10
20
30

¿Cual es el resultado de este código?

```
printf("antes");
if (n > 10) {
    printf("mayor que 10");
} else if (n > 20) {
    printf("mayor que 20");
} else if (n > 30) {
    printf("mayor que 30");
}
printf("despues");
```

¡A la terminal!




n>10
n>20
n>30
1
2
3
10
20
30

¿Y ahora?

```
printf("antes");
if (n > 30) {
    printf("mayor que 30");
} else if (n > 20) {
    printf("mayor que 20");
} else if (n > 10) {
    printf("mayor que 10");
}
printf("despues");
```

¡A la terminal!




¿Preguntas?

Ejemplo en C: número par

```c
#include <stdio.h>

int main(void) {
    int n = 0;
    int r = 0;
    printf("Ingrese un número: ");
    scanf("%d", &n);

    r = n % 2;
    if (r == 0) {
        printf("%d es par\n", n);
    } else {
        printf("%d es impar\n", n);
    }

    return 0;
}
```



A1

Trabajo en equipo #1

Resolución en Parejas (15 minutos)

La universidad requiere un pequeño módulo de validación para la inscripción a un taller avanzado. El sistema debe pedir al usuario:

1. Su **edad** (número entero).
2. Su **promedio académico** (número decimal).
3. La **cantidad de materias aprobadas** (número entero).

El programa debe calcular y mostrar por pantalla:

- Un mensaje de bienvenida indicando la edad recibida.
- Si el estudiante cumple los requisitos para inscribirse: **debe ser mayor o igual a 18 años Y tener al menos 10 materias aprobadas**.
- En caso de cumplir, mostrar: `"Inscripcion aceptada. Promedio registrado: [promedio]"`.
- En caso contrario, mostrar: `"No cumple con los requisitos mínimos."`.

Ayudín

```c
#include <stdio.h>

int main() {
    int entero;
    float decimal;

    // Input: scanf("%d", &entero);
    // Output: printf("Texto %f", decimal);

    return 0;
}
```

Revisión en Parejas (15 minutos)

1. **Estructura base:** ¿Incluye `#include <stdio.h>`, `int main()` y `return 0;`?
2. **Declaración de variables:** ¿Todas las variables tienen su tipo explícito (`int`, `float`) antes de usarse?
3. **Puntos y comas `;`:** ¿Casi todas las líneas de instrucción terminan en ; (excepto `#include`, `main` e `if`)?
4. **Uso de `scanf`:** ¿Utilizaron los especificadores correspondientes (`%d`, `%f`) y el símbolo `&` antes de la variable?
5. **Bloques de código:** ¿Las condiciones `if` / `else` encierran sus bloques entre llaves `{ }`?
6. **Lógica:** ¿La condición lógica (`&&`) evalúa correctamente la edad y las materias?

Tarea del Revisor: Si encuentran un error, **marquen la línea con un círculo** y escriban una nota al pie explicando qué fallaría al intentar compilar o ejecutar.

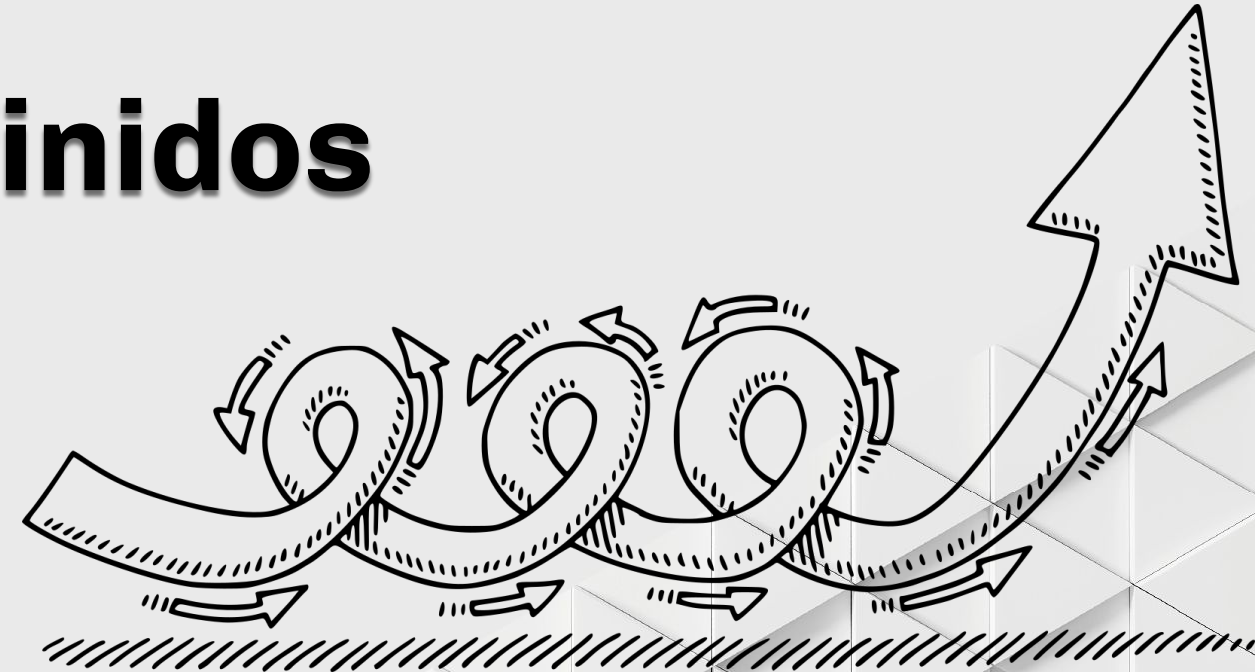

¿Preguntas?

Repetiéndonos

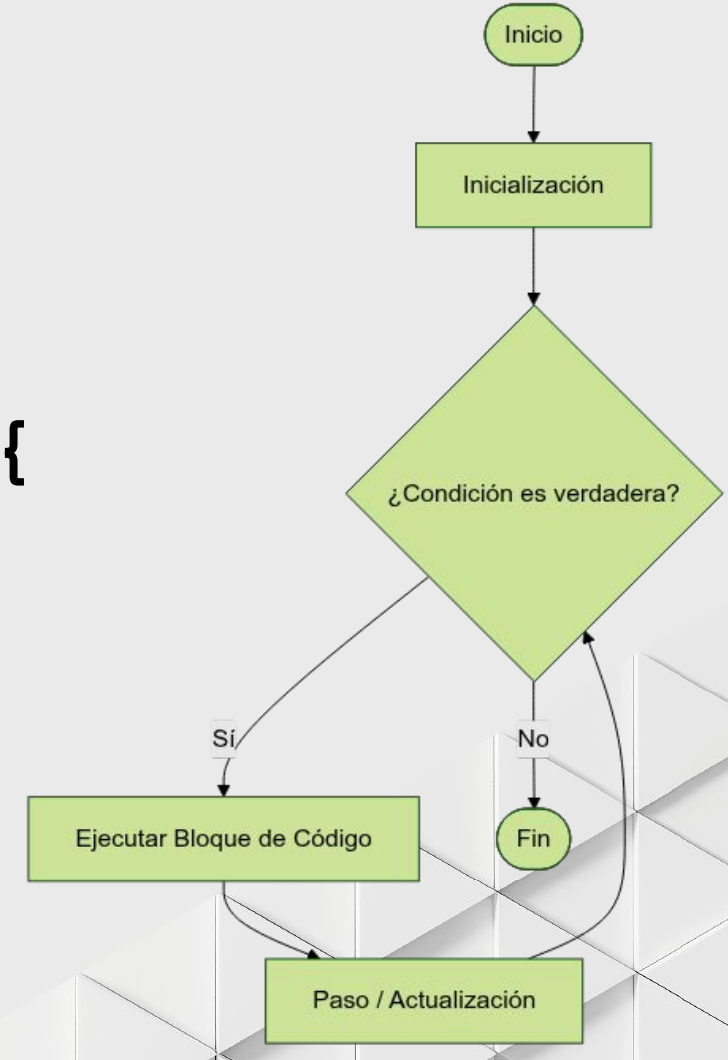


UNRN Universidad Nacional de Río Negro

Lazos definidos

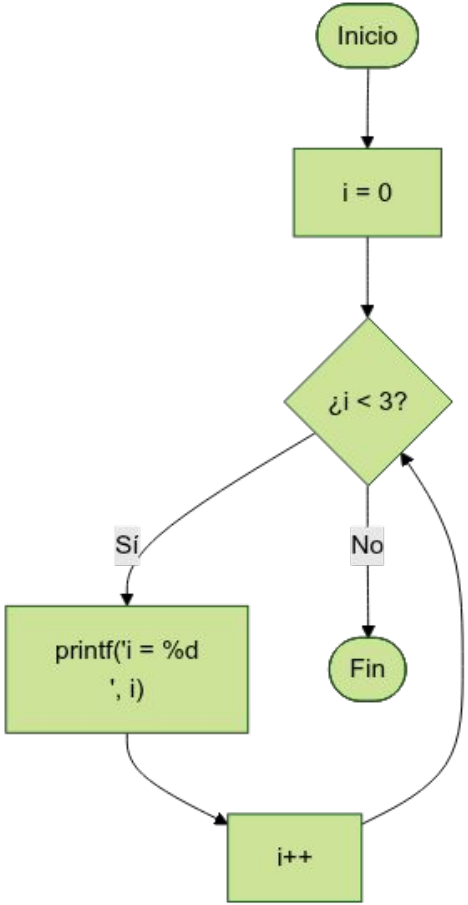


```
for (inicio; condicion; paso) {
    bloque;
}
```



Lazo definido `for`

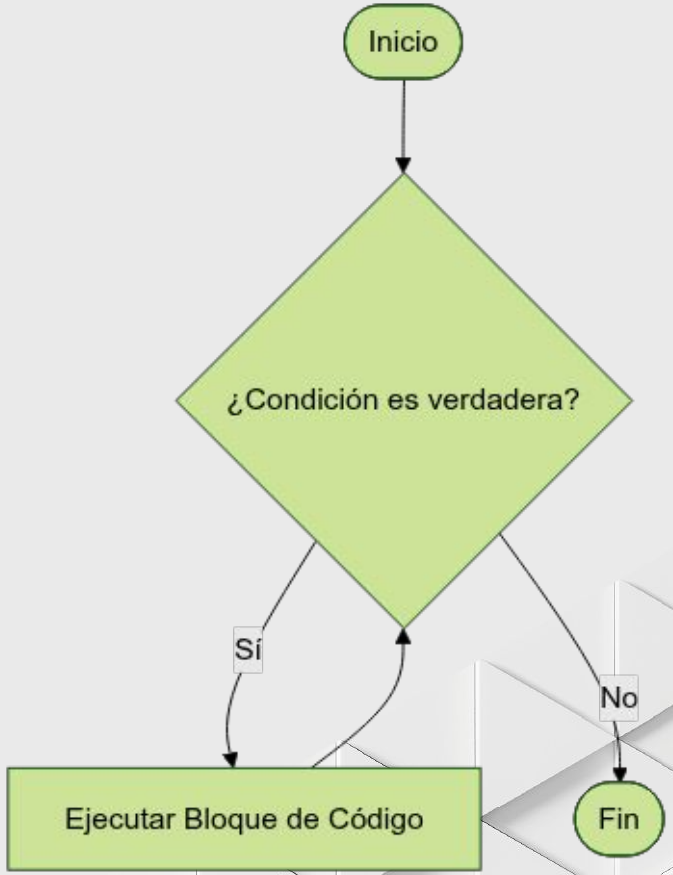
```
for (int i = 0; i < 10; i++) {
    1️⃣
    bloque;
}
2️⃣
```



¡A la terminal!

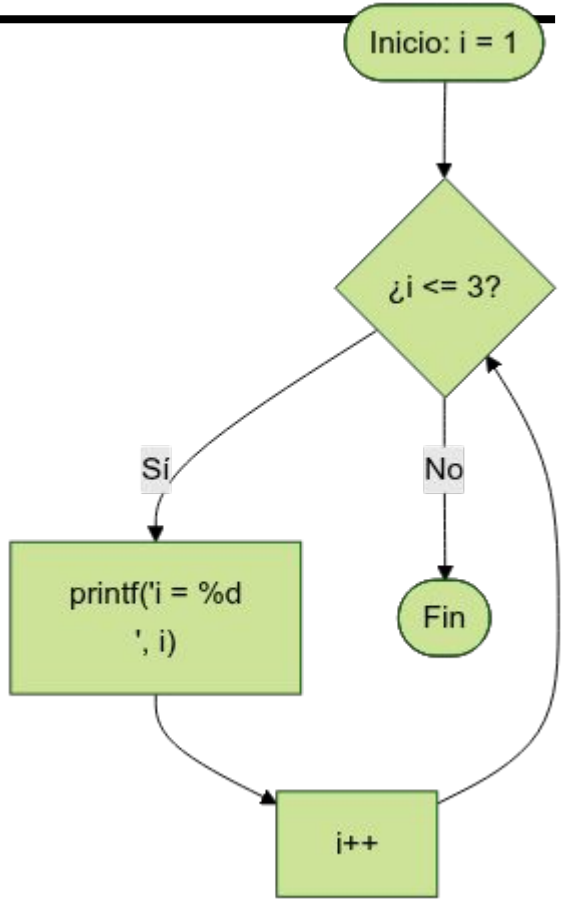


```
while (condición) {
    bloque;
}
```



Lazo indefinido `while`

```
int i = 0;
while(i <= 3) {
    1️⃣
    i++;
}
2️⃣
```



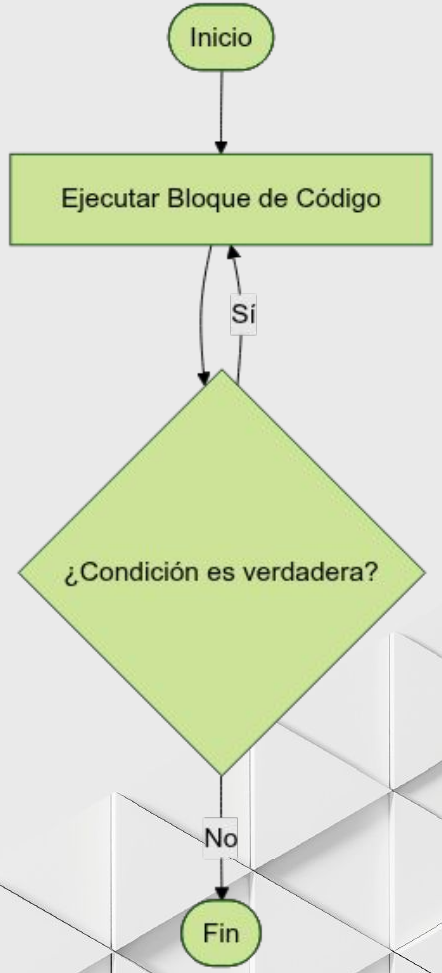
¡A la terminal!



for y while
son intercambiables*

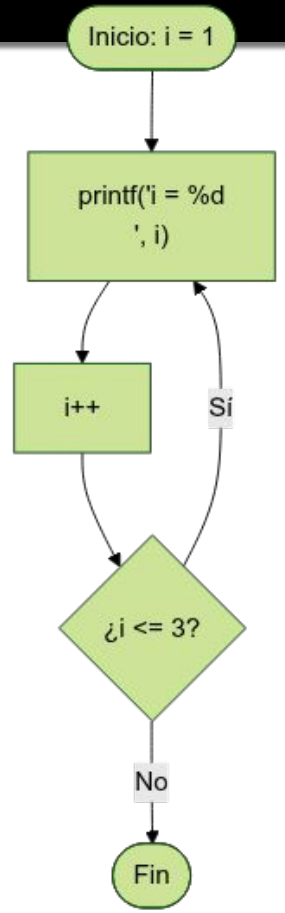
*por lo general

```
do {
    bloque;
} while (condición);
```



Lazo indefinido `do..while`

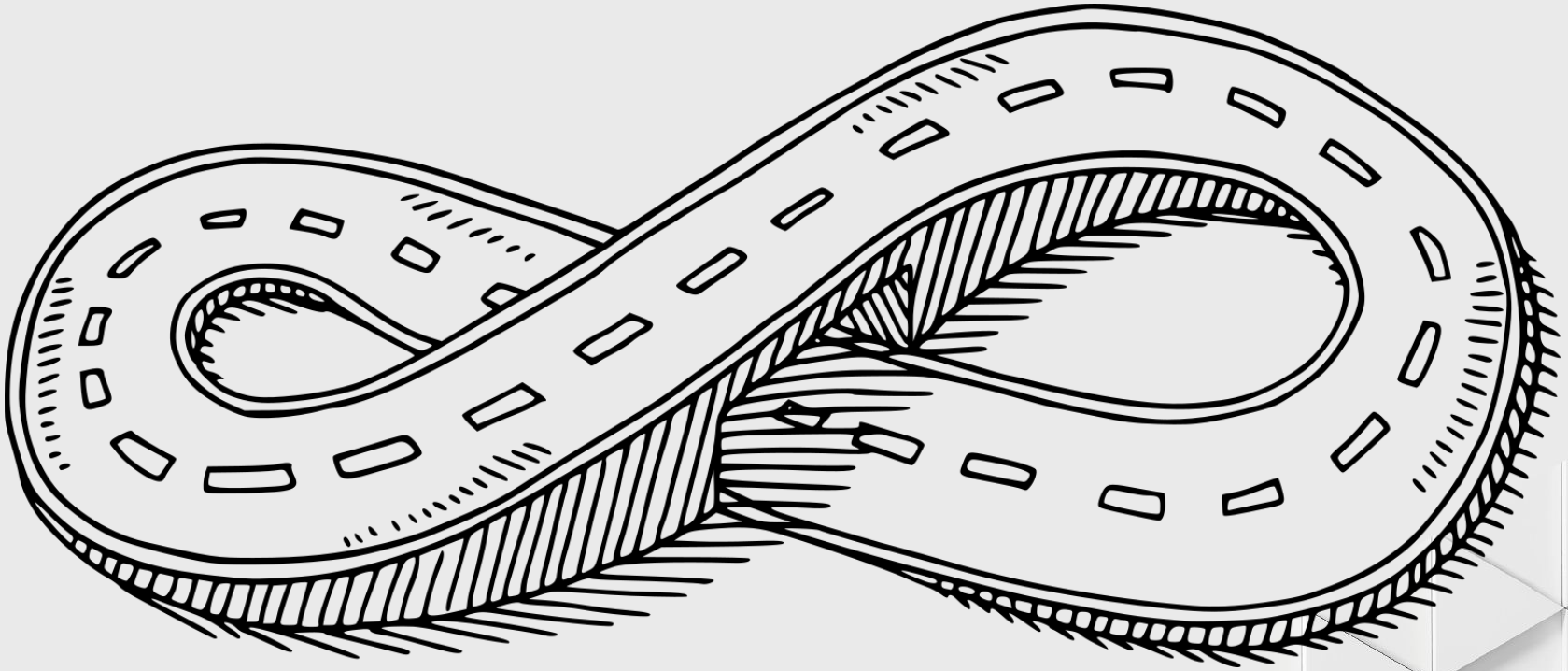
```
int i = 0;
do
{
    1️⃣
    i++;
}
while (i < 10);
2️⃣
```



¡A la terminal!




while (not edge) {
run();
}
do {
run();
} while (not edge);



Lazos más comunes

UNRN Universidad Nacional de Río Negro

De menor a mayor

```
int numero = 0;
while (numero < 10) {
    printf("numero:%d", numero);
    numero = numero + 1;
}
```

Este es uno de los más comunes.

De menor a mayor

```
int numero = 0;
do {
    printf("numero:%d", numero);
    numero = numero + 1;
}
while (numero < 10);
```

Este es uno de los más comunes.

De mayor a menor

```
int numero = 10;
while (numero > 0) {
    printf("numero:%d", numero);
    numero = numero - 1;
}
```

Cuando tenemos menos idea de cuantas veces ejecutar

```
int numero = 10;
bool bandera = true;
while (bandera) {
    printf("numero:%d", numero);
    numero = numero - 1;
    if (numero > 0) {
        bandera = false;
    }
}
```

¡El `if` interno puede ser mucho más complejo!

¡La condición puede ser compuesta!

```
int numero = 10;
while (((numero > 0) && (numero < 20)))
{
    printf("numero:%d", numero);
    numero = numero + 1;
    if (numero % 2 == 0)
    {
        numero = -numero * 2;
    }
}
```

Aunque este ejemplo sea *un poco rebuscado* :-)

Suma de una cantidad abierta de números

```
int numero;
int cantidad = 0;
int suma = 0;
bool activo = true;
while (activo)
{
    printf(“numero %d”, cantidad);
    scanf(“%d”, &numero);
    cantidad = cantidad + 1;
    if ((cantidad > 100) && (numero < 0))
    {
        activo = false;
    }
}
```

¡El `if` interno puede ser mucho más complejo!

Manipulaciones de lazos

UNRN Universidad Nacional de Río Negro

break

```
int i = 0;
while(i < 10)
{
    break;
    i++;
}
printf("i es %d\n", i); 1️⃣
```

`break;`
para cortar el lazo

`continue ;`
para pasar a la siguiente iteración

Los lazos sin `break` y `continue` en su lugar, lazos con bandera

Prefieran usar `while` en lugar de `for`

Deciciones multiples

UNRN Universidad Nacional de Río Negro

```
switch (expresión)
{
    case valor:
        sentencia
    break;
    default:
        sentencia;
}
```



```
switch (expresión)
{
    case valor:
        sentencia
    break;
    default:
        sentencia;
}
```



¡son opcionales!

El no usar break agrupa casos

```
switch (numero)
{
     case 1:
     case 2:
     case 3:
     case 4:
     case 5:
     case 6:
          printf ("numero es >= 1 y <= 6\n");
          break;
}
```

El no usar break agrupa casos

```
int veces = 0;
switch (numero)
{
     case 1:
        veces = veces + 10;
     case 2:
        veces = veces + 10;
     case 3:
         veces = veces + 10;
         break;
}
printf ("veces es %d\n", veces);
```

¿Que valores acepta?
¿Cuál es la salida?

¡A la terminal!




¿Preguntas?

Resolución en Parejas (15 minutos)

El departamento de sistemas de un club deportivo necesita una pequeña aplicación para procesar los tiempos de entrenamiento de sus atletas.

Escriban un programa en C que realice lo siguiente:

1. **Validación de entrada:** Solicite al usuario la cantidad N de atletas a evaluar. El programa debe verificar que N sea un número positivo ($N>0$). Si el usuario ingresa un valor menor o igual a cero, debe volver a pedirlo en bucle hasta que ingrese un valor válido (usando `while` o `do-while`).
2. **Carga de datos y acumulación:** Utilizando un lazo `for`, el programa debe pedir consecutivamente el tiempo en segundos (número flotante) de cada uno de los N atletas.
3. **Cálculo de estadísticas:** Durante o al finalizar la lectura, el programa debe determinar:
    - El **tiempo promedio** del equipo.
    - La **cantidad de atletas** que superaron la marca de calificación (tiempo menor a 60.0 segundos).
4. **Resultado:** Mostrar por pantalla el promedio general y el total de atletas calificados.

Ayudín II

```
// Bucle while (valida mientras la condición sea verdadera)
while (condicion) { ... }

// Bucle do-while (ejecuta al menos una vez)
do {
    ...
} while (condicion);

// Bucle for (inicialización; condición; incremento)
for (int i = 0; i < limite; i++) {
    ...
}
```

Revisión en Parejas (15 minutos)

1. **Inicialización:** ¿Los acumuladores (ej. `suma = 0.0`) y contadores (ej. `calificados = 0`) se inicializan antes de entrar a los lazos?
2. **Validación ($N>0$):** ¿El bucle `while` o `do-while` repite la solicitud correctamente cuando $N<0$?
3. **Sintaxis del `for`:** ¿El bucle `for` tiene sus tres secciones separadas por punto y coma (`;`) y la variable de control se incrementa adecuadamente?
4. **Acumulación e incremento:** ¿Se utiliza correctamente `+=` o `suma = suma + valor` y `i++` dentro de los lazos?
5. **Condición de corte:** ¿El lazo no genera un bucle infinito por olvido de actualización de variables?
6. **Promedio fuera del lazo:** ¿El cálculo del promedio se realiza **después** de completar el bucle `for` (y no en cada iteración)?


¿Preguntas?

{ El jueves lo pasan a la computadora ;-) }

Hasta la próxima

UNRN Universidad Nacional de **Río Negro**